ServiceNow Certified System Administrator study guide



Platform overview & navigation	**7%**	Instance configuration	**11%**
Applications for collaboration	**20%**	Self-service & automation	**20%**
Database management	**27%**	Data migration & integration	**15%**

60 questions in 90 minutes. Database Management alone is more than a quarter of the exam. Expect "choose two" and "choose three" questions; select exactly that many. Use this guide after completing the ServiceNow Administration Fundamentals (SNAF) course on ServiceNow University.

How to use this guide

This guide is a companion to the ServiceNow Administration Fundamentals e-book, not a replacement for it. Everything on the exam is covered in that book. Use this guide to check your understanding after each module and to drill the details the exam likes to test.

> **Watch for:** boxes like this mark distractors and traps that show up again and again.

> **Key point:** boxes like this mark facts you should be able to recall cold.

Do everything you read about in a Personal Developer Instance (PDI). Clicking through a module once is worth more than reading about it three times.

Client vs. server

Knowing where something runs eliminates wrong answers fast.

Client side (the browser)

Controls forms and fields as the user works

- Client Scripts (onLoad, onChange, onSubmit, onCellEdit)
- UI Policies

Server side (the instance)

Controls data no matter how it arrives

- Business Rules
- Data Policies
- Access Control (ACL) scripts
- Flows, Script Includes, transform maps

- Catalog Client Scripts and Catalog UI Policies

A **UI Policy** only affects data entered on a form. A **Data Policy** applies to forms, import sets, and web services alike.

1. Platform overview and navigation 7%

ServiceNow is a cloud platform. Scripting uses **JavaScript**. Response time for a user is the **round trip** between client and server: a **request** plus a **response**.

Interfaces and the Unified Navigation header

ServiceNow is a cloud-based **Application Platform as a Service (aPaaS)** made of three parts: a single enterprise-wide database, a suite of applications, and a development environment. Each **instance** is a full, stand-alone copy of the platform with its own URL. ServiceNow uses a **single-tenant** architecture, so no part of an instance, including the database, is shared with other customers. Companies usually run several instances (Dev, Test, Prod): changes are deployed up the stack and clones copy down it. A **Personal Developer Instance (PDI)** is a free instance for learning and demos.

The main interfaces are the **Now Platform user interface**, **Service Portal**, and the **Now Mobile** apps. The current platform UI is the **Next Experience**, and its main menu bar is the **Unified Navigation header**. An instance with no changes is called a **baseline** implementation.

Header item	What it does
All	Opens the **Application Navigator**: applications, their categories, and modules
Favorites	Items you've marked as favorites
History	Pages you've recently visited, newest first (30 by default)
Workspaces	Single-page workspaces built for a specific kind of work
Admin	Modules for users with the admin role
Contextual app pill	Shows the title of the page you're on; its star adds it to Favorites
Global search	Searches keywords and identifiers across applications and records
Globe icon	The **application scope picker** and **update set picker**: see and switch your current application and update set
Discussions (chat bubbles)	Opens the Discussions sidebar so users can communicate and collaborate
Embedded help (question mark)	Context-sensitive help and articles for where you are, including guided tours
Notifications (bell)	Notification settings: how and when you're notified
User menu (avatar)	Profile, **Preferences** (theme, **time zone**), **Impersonate user**, **Elevate role**, and Log out

Application Navigator

- Each **application** contains categories and **modules**, the links that open screens and tools.
- **Favorites:** create them from a list's List controls menu or a form's Additional actions menu (**Create Favorite**), or with the star on the contextual app pill. Edit a favorite's label with the **pencil** icon.
- Type in the navigator filter for shortcuts: `table_name.list` opens the list, `table_name.form` opens a new form, and `table_name.filter` opens the list without running the query, which is handy for large tables. Typing the suffix in capitals (`.LIST`, `.FORM`, `.FILTER`) opens it in a new browser tab.

Screen types you'll see: **list views** (many records), **form views** (one record), **workspaces**, **dashboards** (reports and Performance Analytics widgets on one screen), **knowledge articles**, and the **Service Catalog**.

Form header

Item	What it does
Additional actions (three lines)	The form context menu: **Save**, **Insert**, **Insert and Stay**, Configure, Export, View, Create Favorite, Copy URL, Copy sys_id, Show XML, History, Reload form. You can also right-click the header.
Paperclip	Manage attachments: add, delete, download
More options (…)	Toggle the **template bar** and toggle annotations
UI Action buttons	Actions on the record, such as Update and Delete
Previous / Next arrows	Step through records when the form was opened from a list

- **Insert** and **Insert and Stay** create a new record as a copy of the one on screen. Insert closes the form; Insert and Stay leaves the copy open.
- Saving: **Save** keeps the form open. **Update** saves an existing record and closes the form. **Submit** saves a new record and closes the form. Changes aren't saved until you do one of these.
- **Templates** fill a group of fields from saved values. Open the template bar from More options (…).
- **Related lists** at the bottom of a form show records in other tables with a one-to-many relationship to this record, such as a user's roles and groups.

Lists and filters

A **list** shows many records; each row is a record and each **column is a field**. A **form** shows one record.

- A filter condition has three parts: **field**, **operator**, and **value**. The operator is always a choice list.
- The fields offered depend on the table, the columns on it, and the user's access rights.
- Build two conditions that must both be true with **AND**, then click **Run**. OR means either can match.

Quick filter technique	How
Show only one value	Right-click a value in the list > **Show Matching** (or Filter Out to exclude it)
Column search	Click the list magnifier, type under a column. `*email` means **contains**, `email%` means starts with, and `%email` means ends with. Text with no wildcard runs a greater-than-or-equal search.
Breadcrumbs	Click a breadcrumb to remove the conditions after it
Natural Language Query	Type a plain-language phrase to filter the list

List view tools

Tool	What it does
List controls menu (three lines)	Switch views, apply saved filters, Group By, Show (rows per page), Refresh List, Create Favorite
Filter icon (funnel)	Opens the **condition builder** (Run, Save, AND, OR, Add Sort)
List search / column search bar	Narrow results by field; the magnifier toggles the column search bar
List personalization (gear)	Choose which columns *you* see, plus wrap text, compact rows, list editing
Actions on selected rows	Act on the records whose checkboxes are selected
Column options menu (three dots on a header)	Sort, Group By, **Bar Chart**, **Pie Chart**, Configure, Import, Export, Update Selected, Update All
List fields menu (right-click a cell)	Show Matching, Filter Out, Copy URL, Copy sys_id, **Assign Tag**
Inline editing	Double-click a cell to edit it from the list
Pagination controls	Total count and page-by-page navigation

Personalize (gear) changes only your view. **Configure > List Layout** changes the list for everyone.

Context menus and tags

Open a context menu by **right-clicking** a list, form header, or column heading, or by clicking its menu icon. Add a tag by right-clicking a record in a list and choosing **Assign Tag**. Tag visibility can be **Everyone**, **Groups and users**, or **Me**.

Dot-walking

A **reference field** stores another record's **sys_id** (a foreign key). **Dot-walking** follows those references to reach related data, for example from an incident to the caller's department name. You can dot-walk in the condition builder, list personalization and list layout, report builders, notification recipients, and scripts (`current.caller_id.department.name`).

2. Instance configuration 11%

Branding, properties, and plugins

System Properties > Basic Configuration UI16 controls the instance look, including: banner image, **browser tab title**, **base theme**, **header background color**, banner text color, **module text color**, and background and divider colors. System Properties > My Company holds the company **banner image** and **banner text**.

> **Watch for:** default calendar, font style, startup sound, animation style, and preferred browser are not Basic Configuration UI16 settings.

Users fix their own date and time display by setting their **time zone** in **user menu > Preferences**. Don't change the instance time zone for one person.

Plugins

Plugins add applications or functionality. Activate them from System Definition > Plugins (or All Available Applications).

- You can't activate a plugin directly in production. Test it on a non-production or **Personal Developer Instance**, then request it for production through Now Support.
- When a plugin depends on others, the system tells you before activation and activates the dependencies.
- Once activated, a plugin **can't be deactivated**. Some offer demo data at activation.

Admin roles, high security, and impersonation

Role	Grants
`admin` (System Administrator)	Access to all system features, functions, and data, because admins can override ACL rules and pass all role checks. For sensitive data such as HR records, ServiceNow recommends custom admin roles instead.
`security_admin`	Elevated role needed to change high security settings and ACLs. Activate it with **Elevate Role** from the user menu.
`impersonator`	Lets a non-admin impersonate other users, but not admin users

Elevation lasts only for your current session. You can drop it two ways:

- **Uncheck the role:** click your user profile icon (avatar) in the top-right corner, select **Elevate Role**, uncheck **security_admin**, and click **Update** (or OK).

- **Log out:** log out and log back in, or wait for the session to time out. Elevated privileges reset automatically when the session ends.

Common security modules: System Properties > Security, System Security > High Security Settings, and System Security > Access Control (ACL).

Impersonation

- Start it from the **user menu in the banner**. Use it to test what a user sees.
- Return to your own account with **End Impersonation**.
- Impersonation is logged when the `glide.sys.log_impersonation` system property is true.

Forms, fields, and UI Policies

Tool	Open it from	Use it for
Form Layout	Context menu > Configure > Form Layout	Slushbucket: add, remove, and reorder fields and sections
Form Design	Context menu > Configure > Form Design	Drag-and-drop designer; can also create new fields

Both can add fields and change field order. The Form Designer has three main parts: the **page header**, the **field navigator**, and the **form layout**.

- **Sections** display as tabs (when the user has tabbed forms on) once you **give the section a name**.
- To change a field's label, right-click the label and choose **Configure Label**, or edit it from **Configure Dictionary**.
- Templates are opened from the form header's More options (...) icon.
- **Form Layout** is the traditional slushbucket tool; **Form Design** is the drag-and-drop (WYSIWYG) designer. Both build form **views**, which can differ by persona.

Field types

Need	Type
Free text with letters, numbers, special characters	String
Drop-down of predefined options	Choice
Look up a record in another table	Reference
Checkbox	True/False
Date and time	Date/Time

UI Policies

A UI Policy is the **no-code** way to make fields **mandatory**, **read-only**, or **hidden** based on conditions. Example: make Resolution code mandatory when State is Resolved, and hide a checkbox unless the user has a role. UI Policies run on the client, after Client Scripts, and only for form interactions.

3. Configuring applications for collaboration

20%

Users, groups, and roles

Object	Definition	Table
User	A person who can log in	`sys_user`
Group	A set of users who share a common purpose (a team)	`sys_user_group`
Role	A collection of permissions	`sys_user_role`

- Best practice: assign roles to **groups**, then add users to groups.
- Roles can contain other roles, and groups can contain other groups.
- Relationship tables: Group Member [`sys_user_grmember`] links users to groups, Group Role [`sys_group_has_role`] links groups to roles, and User Role [`sys_user_has_role`] links users directly to roles (possible but not recommended).
- A **persona** (requester, fulfiller, approver) describes a user by the work they do. Personas are implemented with roles.
- Manage users, groups, and roles in the **User Administration** application.
- Setting up a new IT worker: **create the user**, then **add them to each group** whose work they'll do.

If an IT user only sees self-service, they're missing the `itil` role, usually because they aren't in a group that grants it. A manager sees nothing in **My Groups Work** if they aren't a *member* of those groups; being listed as the group's manager doesn't count.

Tasks, assignment, and the activity stream

Task work can be assigned to **users** and **groups**. **Assignment rules** automatically decide which group gets which kind of task.

Benefits of assigning to a group: members can pick work by SLA urgency or highest priority, and the group can route work by on-call schedule, skills, and availability.

- The **Activity** section (activity stream) shows the most recent updates and the history of work on a record. Use its **funnel icon** to show only work notes.
- On a list, the activity stream icon opens a frame with live updates for records you're watching.
- **My Approvals**: right-click an approval in the list and choose Approve or Reject.

UI Actions

UI Actions add **buttons, links, and context menu items** to forms and lists.

Form UI Actions	List UI Actions
Form buttons, form context menu items, form links	List buttons, list context menu items, list choices, list links

Notifications

Notifications inform users about activity. They're created in **System Notification > Email > Notifications** and can be triggered by record inserts and updates or by **events**.

Tab	Defines
When to send	Record inserted or updated, or event fired; conditions
Who will receive	Users, groups, and **users/groups in fields**. Dot-walk from Caller to Caller.Manager to include the caller's manager.
What it will contain	Subject, message, and an **email template** for reusable content like a company banner

Reporting and Performance Analytics

Create reports from **Reports > View / Run**, from **Reports > Create New**, or by right-clicking a list column header.

Report designer section	Purpose
Data	Source table or report source, and filters
Type	Visualization: list, bar, horizontal bar, pie, donut, semi-donut, speedometer, line, and more (25+)
Configure	**Grouping and calculations**
Style	**Colors, titles, legend layout**

- Click **Run** to see changes without saving.
- Best practice: copy a report before editing it, using the dropdown next to Save (**Insert and Stay** / Save as).
- Reports on Service Catalog fulfillment of a specific item use **Requested Item [sc_req_item]**.
- Pull in data from a related table with **dot-walking**. If the same complex data set is reused often, give users a **report source**. If data lives in two tables and dot-walking can't reach it, create a **database view**.
- Share reports with yourself, groups, or everyone; schedule them to be emailed; add them to dashboards.

Performance Analytics

Use Performance Analytics when you need data **trended over time** and **forecast** into the future, using scheduled snapshots, KPIs, and dashboards. Standard reports show the data as it is right now. Learn it through a Performance Analytics course on Now Learning.

4. Self-service and automation 20%

Service Catalog

Build	When
Catalog item	One orderable good or service. Options such as color or RAM are **variables**.
Order guide	Several related items ordered together in one request (new employee: laptop, monitor, phone, business cards)
Record producer	A catalog form that creates a record in another table, like an **incident** from a "Get Help" item. Skips the request and requested item.
Content item	A catalog item that provides information (for example, a link or article) instead of a good or service
Variable set	A reusable group of variables added to many items (four shared mandatory fields across 80 items)

Structure: a **Catalog** [`sc_catalog`] contains **Categories** [`sc_category`], which can have subcategories, and categories hold **Catalog Items** [`sc_cat_item`]. A company can have many catalogs. Shoppers use **Self-Service > Service Catalog**; admins use the **Service Catalog** application.

Use **Catalog Builder** to create and publish catalog items and record producers from **templates** (a template is a set of pre-made decisions that simplifies building an item).

Role	Can do
`admin`	Everything in the Service Catalog, including related scripting (business rules, UI macros)
`catalog_admin`	Fully manage the Service Catalog application, without the scripting functions
`catalog_manager`	Update a specific catalog they manage and assign its editors and managers
`catalog_editor`	Edit a specific catalog and its categories and items
`catalog_builder_editor`	Create and maintain items in Catalog Builder using templates

- Request flow: **Request (REQ, sc_request) → Requested Item (RITM, sc_req_item) → Catalog Task (SCTASK, sc_task)**.
- One request has many requested items; one requested item has many approvals and many catalog tasks.
- When every catalog task for a requested item is complete, the requested item closes. When every requested item is complete, the request closes.
- Control who can see an item with **user criteria** (Available for / Not available for).
- One item needed in two catalogs? Create it once and **publish it to both catalogs**.

A manager approval is skipped when the requester's user record has **no manager**. When testing an approval flow: make sure the latest flow is active, impersonate the requester, and confirm the requester has a manager. On instances using legacy workflows, the approval threshold lives in the **Service Catalog Request** workflow.

Flow Designer

Flows automate business logic such as approvals, tasks, notifications, and record operations with little or no code. Open it from **Process Automation > Flow Designer**.

- A flow starts with a **trigger** (record, schedule, or application) and a **condition**.
- **Actions** do the work. Common core actions: Create Record, Look Up Record, Ask for Approval, Wait for Condition.
- Each action's output becomes a **data pill**. Drag data pills into later steps to reference record data. If a later action changes a data pill's value, the new value is used for the rest of the flow.
- Flows handle multi-level approvals with notifications at each level.

Advantages: less manual scripting, reduced technical debt, and smooth third-party integration through **IntegrationHub spokes**. Spokes are available from the **ServiceNow Store**. **Process Automation Designer** combines cross-enterprise workflows into one unified process.

Knowledge Management

Knowledge Management lets the organization create, categorize, review, approve, and browse information in one shared place.

- Access is controlled with **user criteria** on the knowledge base's **Can Read** and **Can Contribute** lists. Articles can also have their own Can Read user criteria to narrow access further.
- If a knowledge base has no user criteria, **all users can read it** by default. Newer instances can block this with a security property, so always set Can Read.
- With **Match All** selected, a user must meet every condition (company AND department AND group).
- Importing articles requires **Can Contribute** access.
- Check a specific user's access with **User Criteria Diagnostics**.
- **Workflows** control publishing and retiring articles. Article states: Draft, Review, Published, Retired.
- `knowledge_admin` manages multiple knowledge bases; a knowledge manager owns a specific one.
- Structure: a **Knowledge Base** [`kb_knowledge_base`] has an **owner** and a **publish** and **retire workflow**. It is divided into **categories** [`kb_category`]; a category belongs to one knowledge base. Each **article** [`kb_knowledge`] belongs to one knowledge base and one category.
- Articles can include formatted text, images, video, and code, and can be written in the editor or imported from a Word document.
- Feedback and continuous improvement: star ratings, helpful votes, comments, flagging an article, and creating an incident from an article.

Virtual Agent delivers help, knowledge, and common tasks through a conversational messaging interface.

Scripting basics

A **Business Rule** runs on the server when a record is displayed, inserted, updated, deleted, or when a table is queried. Its **When** options are **before**, **after**, **async**, and **display**. A **Client Script** runs in the browser.

5. Database management 27%

The biggest domain. Spend real time here, and build tables, relationships, and ACLs in your PDI.

Tables, fields, and relationships

A table holds records (rows); each **field** (column) holds one piece of data for a record. The primary operations on tables are **create, read, write, and delete**.

Term	Meaning
Core table	Ships with the platform (User, Task, Incident)
Custom table	Created by you (`u_` prefix in global; scope prefix such as `x_` or `sn_hr_` in a scoped app)
Base table	Not extended from any other table (Task)
Parent table	Has tables extended from it; may itself extend another table
Child table	Extends another table and inherits its fields

A table can't be both core and custom. Any other combination is possible.

Worked example: Incident extends Task, and Super Incident extends Incident. Task is a **base** and **parent** table. Incident is both a **child** and a **parent**. Super Incident is a **child**.

For a new app whose tickets get assigned to a team, **extend Task** and add your own fields, so you inherit assignment, state, and work notes.

Relationships

Relationship	Example
One-to-many	One request has many requested items (a reference field)
Many-to-many	Vendors sell many products; products come from many vendors. Related lists show on both tables.
Extension	Incident extends Task
Database view	Joins two tables for reporting

- Create tables with **System Definition > Tables** or **Tables & Columns**.
- See a table's relationships with the **schema map**: System Definition > Tables, open the table, Related Links > Show Schema Map.

- **Dictionary tables** describe the database itself: Table [`sys_db_object`] has a record for every table, and Dictionary Entry [`sys_dictionary`] has a record for every field. Dictionary overrides change a field's properties on a child table.
- Every table and field has a **name** (used in code, no spaces) and a **label** (shown in the UI).
- **sys_id** is the primary key on nearly every table. A reference field stores another table's sys_id.
- A new custom table gets four ACLs automatically: **create, read, write, delete**.

Access Controls (ACLs)

ServiceNow is **deny-first**: ACLs don't deny access, they **grant** it under set conditions. ACLs are stored in Access Control [`sys_security_acl`] and managed in the **System Security** application. They mainly protect records and fields for each operation (create, read, write, delete), and can also protect UI pages, client-callable scripts, and processors. Permissions are set with **roles**, **conditions**, and a **script** that sets `answer` to true or false. Creating or changing an ACL requires elevating to `security_admin`.

Name	Secures
`incident.None`	All records on the table (row level)
`incident.*`	All fields on the table
`incident.short_description`	One field

Let itil users create incidents: Name `incident.None`, Operation **create**, Role **itil**.

> - Table (row) ACLs are evaluated before field ACLs, so a user must have access to the table before field access is considered.
> - If a row rule and a field rule both apply, **both must pass**.
> - Table ACLs go from specific to general: the table, then its parent table, then the wildcard (`*`).

A user who can see a module but gets a blank page on Create New likely has read but not create access on that table.

Data Policies

Data Policies enforce mandatory and read-only rules on the **server**, for every way data arrives: forms, import sets, and web services. Use them for rules like "Short description is mandatory on insert everywhere" or "Number is read-only for all users." A Data Policy can also be applied as a UI Policy on forms.

CMDB and CSDM

Table	Role
`cmdb`	Base Configuration Item table, top of the CMDB hierarchy
`cmdb_ci`	Configuration Item table; CI classes extend it (cmdb_ci_server, cmdb_ci_computer)
`cmdb_rel_ci`	Relationships between CIs
`cmdb_rel_type`	The relationship types, such as Depends on::Used by

- Defining **suggested relationships** between CI classes makes relating CIs easier, because the platform suggests which types fit.
- The **CI dependency view** graphically shows a CI's infrastructure relationships. Right-click a relationship line and choose **View Relationship Form**; right-click a CI for options like View Form and View Related Tasks. Service desk agents use it to trace from a service to the CIs with active issues.
- On a CI form, **Related Items** shows the other CIs it's related to.
- Three ways to populate the CMDB: **manual entry** (always needed sometimes, but slow and error-prone), **integrations** that import data, and **automated discovery** (ServiceNow Discovery, Service Mapping, or third-party tools such as Microsoft SCCM). Each defined source is a **discovery source**.
- The **Identification and Reconciliation Engine (IRE)** uses identification rules and reconciliation (data precedence) rules to match incoming CIs and prevent duplicates. Identification rules say which attributes uniquely identify a CI; reconciliation rules say which source is trusted at the record and field level.
- **CI Class Manager** is the main tool for CMDB classes (a class is a CMDB table) and IRE rules.
- **CMDB Data Manager** creates and manages CI policies. The **CMDB Health Dashboard** measures Completeness, Compliance, and Correctness. **CMDB Query Builder** queries CIs and relationships with drag and drop.

The **Common Service Data Model (CSDM)** is the standard, shared set of service definitions across ServiceNow products. It's the blueprint for mapping your IT services into ServiceNow and supports service-level reporting.

6. Data migration and integration 15%

Import Sets

Import data with the **System Import Sets** application. The steps are: **load data**, **create a transform map**, **transform**, and **clean up** the import set table.

1. **Load Data** (first step) brings the source into an **import set row table**, the staging area.
2. A **transform map** defines where incoming columns go on the target table (field mapping).
3. **Transform** copies data from staging to the target.

Mapping tool	When
Automatic mapping (Auto Map Matching Fields)	Source and target field names match
Mapping Assist	Map source fields to target fields yourself in a slushbucket-style interface

> **Coalesce** fields are the unique key. Match found: the record is updated. No match: a new record is inserted. **No coalesce field**: every row is inserted as new, and nothing is updated.

A **data source** [`sys_data_source`] tells the platform how to connect to external data and load it into an import set table. Types include files (CSV, Excel, XML, local or from a network server), **JDBC**, **LDAP**, and **REST**. Target tables can be in the current scope, in global, or in other scopes that allow write access.

Record	Table	Purpose
Data source	`sys_data_source`	Where and how to get the data
Transform map	`sys_transform_map`	Import set table to target table
Field map	`sys_transform_entry`	Field-by-field mapping inside a transform map
Scheduled data import	`scheduled_data_import`	Runs an import automatically on a schedule

A **robust transform** is a more advanced transform used mainly for the CMDB: it can load several target tables and create relationships between the imported records.

Best practice: plan time before importing to clean out obsolete or inaccurate data.

Update Sets

Update sets capture configuration changes so you can move them between instances.

1. Create a named update set and make it current.
2. Make your changes, then mark the set **Complete**.
3. On the target: retrieve, **preview**, then **commit**.

> - Never use the **Default** update set to move changes between instances.
> - Update sets carry configuration, not task or process data. Not captured by default: **data records** and **homepages and content pages**. Add a homepage manually (Homepage Admin > Pages, right-click > Unload Portal Page) or a dashboard with **Unload Dashboard**.
> - Keep update sets small to reduce conflicts and make changes easier to review.
> - Moving several at once? **Batch** them.
> - Move data with import sets or XML export and import, not update sets.

Table names to memorize

The exam asks for exact names. Type each one followed by `.list` in your PDI.

Label	Table
User	`sys_user`
Group	`sys_user_group`
Role	`sys_user_role`
Task	`task`
Request / Requested Item / Catalog Task	`sc_request` / `sc_req_item` / `sc_task`
Catalog Item	`sc_cat_item`
Base CI / CI / CI relationship	`cmdb` / `cmdb_ci` / `cmdb_rel_ci`
System Properties / Business Rules / Client Scripts	`sys_properties` / `sys_script` / `sys_script_client`
Group Member / Group Role / User Role	`sys_user_grmember` / `sys_group_has_role` / `sys_user_has_role`
Table / Dictionary Entry	`sys_db_object` / `sys_dictionary`
Access Control	`sys_security_acl`
Catalog / Category	`sc_catalog` / `sc_category`
Knowledge / Knowledge Base / Category	`kb_knowledge` / `kb_knowledge_base` / `kb_category`
CI relationship type	`cmdb_rel_type`
Data source / Transform map / Field map	`sys_data_source` / `sys_transform_map` / `sys_transform_entry`

Off the top of your head

Say the answer out loud, then open it to check.

- Insert vs. Insert and Stay?

Both create a copy of the current record. Insert closes the form; Insert and Stay keeps the copy open.

- Which table holds a record for every table? For every field?

  sys_db_object for tables, sys_dictionary for fields.

- Open a big table's list without running the query?

  Type table_name.filter in the navigator.

- Three parts of a filter condition?

  Field, operator, value.

- Column search for "contains"?

  Start the term with `*`.

- Where do you find the update set picker?

  The globe icon in the banner.

- Role needed for high security settings, and how do you drop it?

  security_admin, elevated for the session. Drop it two ways: open your user profile icon (avatar) > Elevate Role, uncheck security_admin, and click Update (or OK); or log out and back in, or let the session time out. Elevated privileges reset when the session ends.

- How do you return from impersonation?

  End Impersonation from the user menu.

- Group vs. role?

  A group is a set of users; a role is a collection of permissions. Assign roles to groups.

- Report designer section for grouping? For colors and titles?

  Configure. Style.

- Trend and forecast data over time?

  Performance Analytics.

- Laptop, phone, and badge in one request?

Order guide.

- Catalog form that creates an incident?

  Record producer.

- Same four variables on 80 items?

  One variable set added to every item.

- Why would a manager approval be skipped for one requester?

  The requester's user record has no manager.

- Where does action output live in a flow?

  In data pills.

- Knowledge base with no user criteria: who can read?

  All users, by default.

- Tool to check one user's knowledge access?

  User Criteria Diagnostics.

- Business Rule When options?

  Before, after, async, display.

- ACL to let itil users create incidents?

  `incident.None`, operation create, role itil.

- No-code mandatory field on forms only? On all data entry?

  UI Policy. Data Policy.

- Three key CMDB tables?

  cmdb, cmdb_ci, cmdb_rel_ci.

- Blueprint for mapping IT services into ServiceNow?

  CSDM.

- What happens with no coalesce field?

  Every row is inserted as a new record.

- Two things not in an update set by default?

  Data records and homepages.

- Moving several update sets at once?

  Batch them.